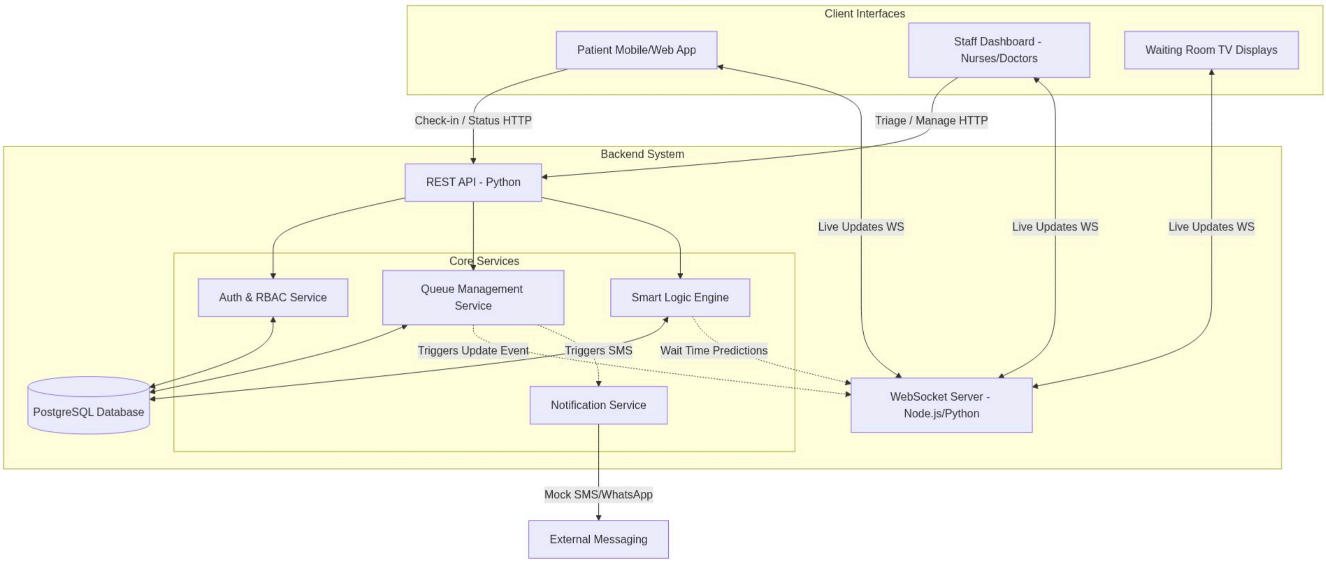
Digital Queue Management System - Backend Architecture & ERD

Based on our project proposal, here is the backend architecture design, database entity-relationship diagram (ERD), and the smart queue logic flowchart. This design is sophisticated enough to address the dual-track priority, real-time updates, and KPI tracking (T1, T2, T3) while remaining feasible for a 7-week development sprint.

1. Backend System Architecture

The architecture separates the RESTful operations (check-in, triage) from the real-time WebSocket connections (live queue updates), ensuring scalability and responsiveness.



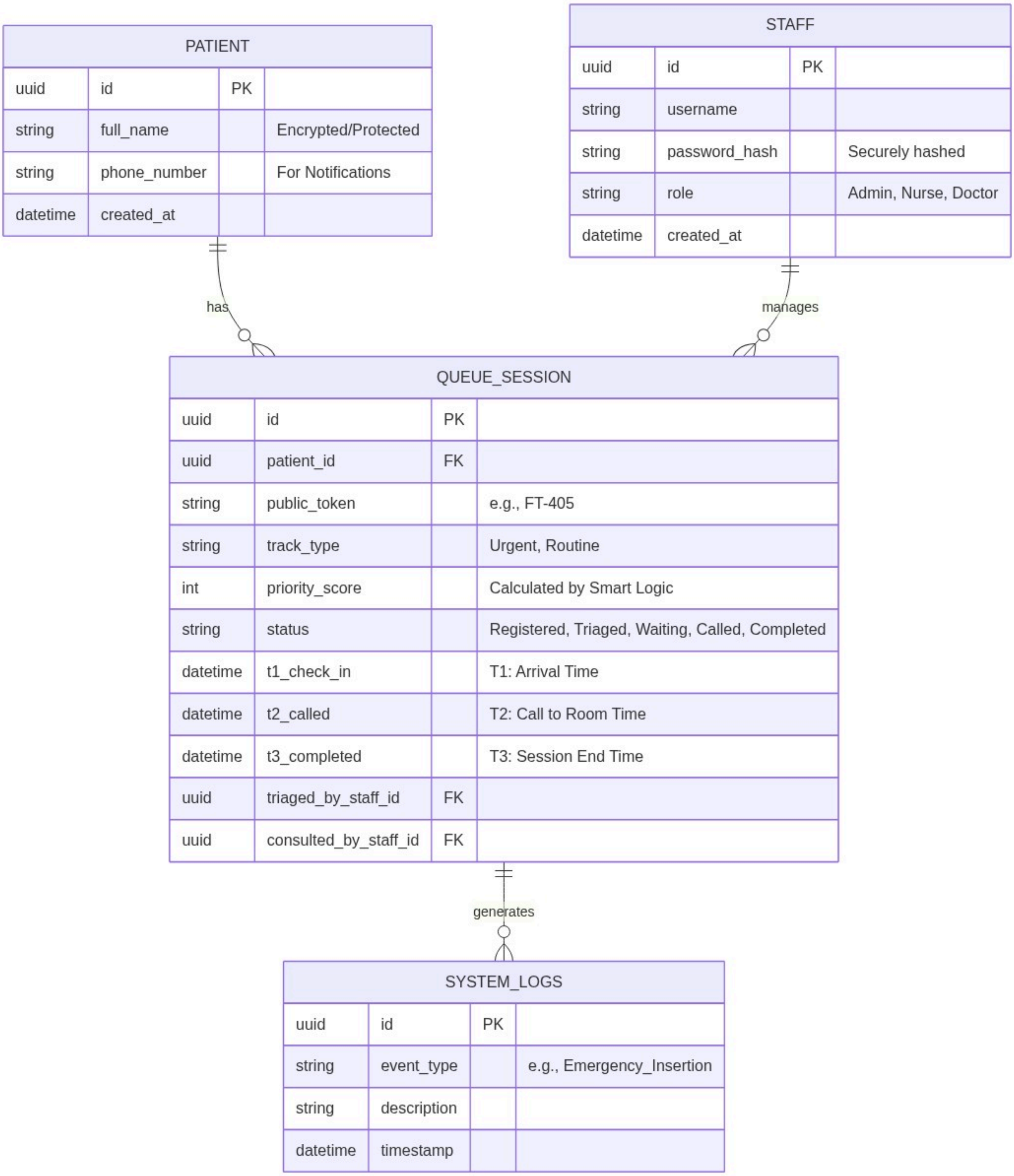
Architecture Components:

- **REST API (Python):** Handles standard requests like user authentication, patient registration, and nurses submitting triage forms. Python allows rapid development of these endpoints.
- **WebSocket Server:** Keeps open, two-way connections with patient phones and waiting room displays to push live queue updates instantly without polling.
- **Smart Logic Engine:** The core algorithm that recalculates estimated wait times dynamically, prioritizes the urgent track, and ensures the routine track doesn't suffer from "starvation" (waiting indefinitely).
- **Notification Service:** A decoupled service to handle SMS/WhatsApp mock notifications.

---

2. Entity Relationship Diagram (ERD)

The database schema is designed in PostgreSQL to securely handle role-based access, patient privacy (using tokens), and track the essential KPIs (`T1`, `T2`, `T3`) for evaluating success.



Database Highlights:

- **Privacy by Design:** The `PATIENT` table stores PII (Personally Identifiable Information) securely, while the `QUEUE_SESSION` table relies entirely on the `public_token` (e.g., FT-405) for public displays.
- **KPI Tracking:** `t1_check_in`, `t2_called`, and `t3_completed` are native columns in the session table, making it trivial to calculate your core metric: `True wait time = T2 - T1`.

---

3. Smart Logic Engine & Dual-Track Flow

This flowchart illustrates how the system processes patients, sorts them into the dual-track queue, and protects them from starvation.